Lecture 24: The thinking revolution

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Explain **chain-of-thought prompting** — why intermediate reasoning steps improve LLM performance
2. Describe **how reasoning models are trained** via reinforcement learning on verifiable rewards
3. Explain the **mechanics of test-time compute scaling** — what happens inside a reasoning model at inference
4. Analyze **why thinking works**: the relationship between token generation and computation
5. Evaluate the evidence: is longer "thinking" genuine reasoning or sophisticated pattern completion?

Welcome back

The last content week

This is our final week of new material. Three lectures remain:

- **Today**: The thinking revolution — how and why reasoning models work
- **Wednesday**: Agents, tools, and the agentic era
- **Friday**: The reckoning — society, safety, and what comes next

Final project reminder

Final Project presentations are **March 9** (next Monday). Submit all materials before class.

The biggest shift since transformers

A new scaling axis

Every model we've studied — from word embeddings to GPT — improved primarily by making **training** bigger: more data, more parameters, more compute during training. In late 2024, a new paradigm emerged: **test-time compute scaling** — spending more compute *at inference* to improve results.

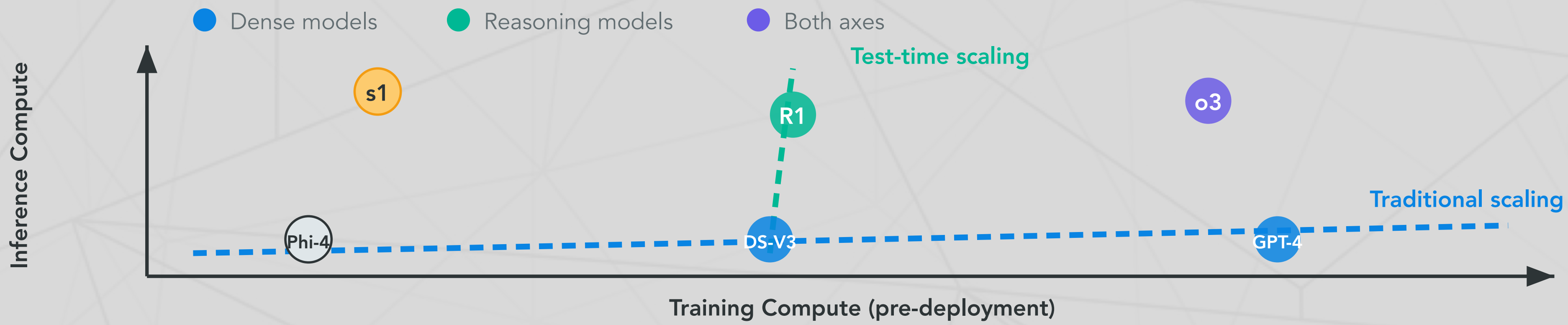
The key insight

A smaller model that "thinks longer" can outperform a larger model that answers immediately. This decouples model quality from model size in a way that changes everything about how we build and deploy AI.

Two scaling axes

Training compute vs. inference compute

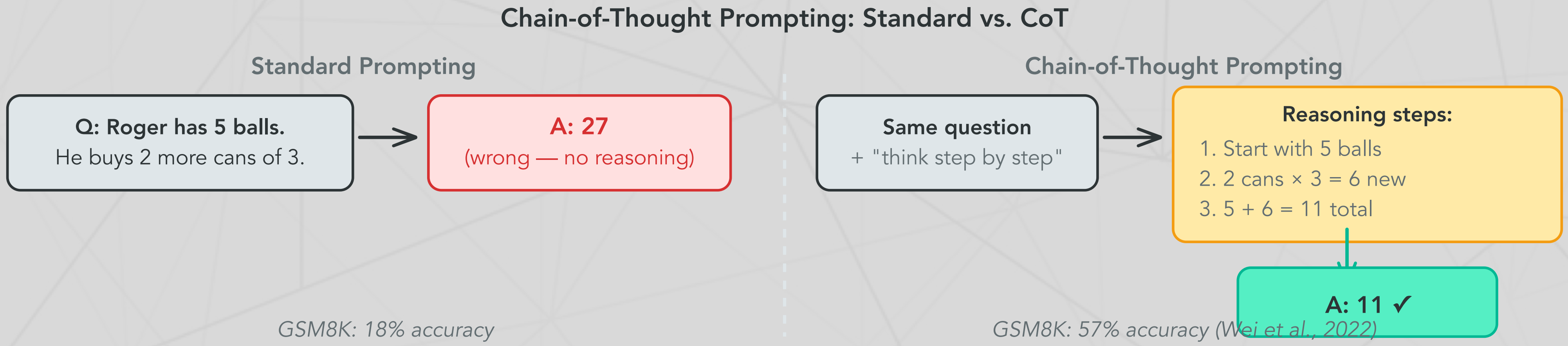
	Training compute	Inference compute
When	Before deployment (once)	At query time (every call)
What improves	Base knowledge, capabilities	Reasoning depth on hard problems
Scaling law	Kaplan et al. (2020)	Snell et al. (2024) — **new**



Chain-of-thought prompting

Wei et al. (2022, NeurIPS) — where it all started

<u>Chain-of-thought (CoT) prompting</u> showed that LLMs perform dramatically better on reasoning tasks when prompted to **generate intermediate steps** before answering. This requires no model changes — just different prompting.



Why does this work?

Each generated token is a **computation step**. When a model writes "2 cans × 3 = 6," it's not just outputting text — it's performing the multiplication via its forward pass and storing the result in context for subsequent tokens. More tokens = more serial computation = harder problems solvable.

Why intermediate steps help: the computational argument

Transformers are constant-depth circuits

A transformer with L layers performs L sequential computation steps per token. For a 100-layer model answering in one token, you get **100 steps of computation** — regardless of problem difficulty.

But if the model generates N intermediate tokens first, it gets $L \times N$ **steps** — the entire model runs once per token, and each token can attend to all previous tokens.

The formal connection

Merrill & Sabharwal (2024) proved that constant-depth transformers are limited to problems in the complexity class TC^0 (constant-depth threshold circuits). But with a chain-of-thought of length T, a transformer can simulate T steps of any Turing machine — making it **Turing-complete**.

In plain language: without CoT, transformers literally *cannot* solve certain problems no matter how large. With CoT, they can solve *anything* (given enough tokens).

From prompting to training: the evolution

The evolution

Year	Technique	Key idea
2022	Chain-of-thought prompting	Prompt the model with reasoning examples
2023	Tree-of-thought, self-consistency	Generate multiple paths, pick the best
2024	**Reasoning models** (o1)	Train the model via RL to reason on its own
2025	Adaptive thinking (Claude 4.x)	Model decides *when and how much* to think

The conceptual leap

CoT prompting (Wei et al., 2022) showed that intermediate reasoning helps. Reasoning models take this further: instead of *prompting* the model to reason, you **train it via reinforcement learning** on verifiable rewards (correct math, passing code tests). The model learns to generate its own internal reasoning traces — and these traces can be far more effective than human-written examples.

How reasoning models are trained

The RL training loop

The key innovation: instead of supervised learning (human writes the reasoning), use **reinforcement learning** where the model discovers *its own* reasoning strategies:

1. **Sample**: Model generates multiple complete solutions (reasoning + answer) for each problem
2. **Verify**: Check which answers are **correct** (math: exact match; code: passes unit tests)
3. **Update**: Reinforce reasoning patterns that led to correct answers; suppress those that didn't
4. **Repeat**: Over millions of problems, the model learns which reasoning strategies work

The crucial requirement

This only works for **verifiable** tasks — problems where we can automatically check correctness. Math, coding, and formal logic have clear right/wrong answers. Open-

GRPO: how DeepSeek-R1 learns to reason

Group Relative Policy Optimization

DeepSeek-R1 uses **GRPO** — a simpler alternative to PPO that doesn't need a separate value model:

1. For each problem, generate a **group** of G candidate solutions (e.g., $G = 64$)
2. Score each: correct answer → reward $+1$, wrong → reward 0
3. Compute **relative advantage**: how much better/worse than the group average?
4. Update the model to increase probability of above-average solutions

Concrete example

```
Problem: "What is 17 × 23?"
  Solution 1: "17 × 23 = 17 × 20 + 17 × 3 = 340 + 51 = 391" √  → reinforce
  Solution 2: "17 × 23 = 17 × 25 - 17 × 2 = 425 - 34 = 391" √  → reinforce
  Solution 3: "17 × 23 = 300 + 91 = 391" √                       → reinforce (less)
```

What emerges from RL training

DeepSeek-R1-Zero: zero supervised fine-tuning

R1-Zero was trained with RL only — no human-written reasoning examples at all. The reward signal was *solely* whether the final answer was correct. Yet the model spontaneously developed:

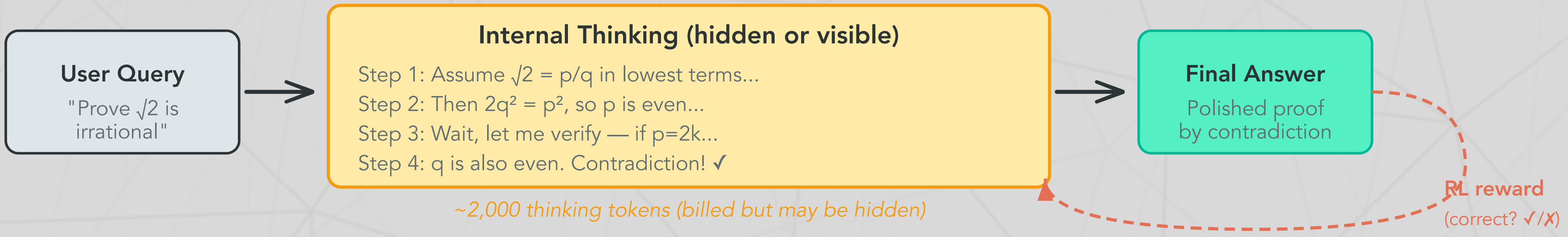
- **Self-verification**: "Let me check this... wait, that's wrong"
- **Backtracking**: "Actually, I should try a different approach"
- **Structured reasoning**: Breaking problems into numbered sub-steps
- **Reflection**: "This seems too easy, let me double-check"

Why this is remarkable

These reasoning strategies were **never demonstrated** to the model. They emerged purely because they lead to more correct answers. The model "discovered" that doubting itself, re-examining its work, and trying alternative approaches is useful — through optimization pressure alone.

AIME 2024: jumped from **15.6% → 71.0%** (pass@1) with RL only.

The reasoning model pipeline



What happens at inference

1. **User sends a query** — the model begins generating "thinking tokens"
2. **Internal reasoning** — the model works through the problem step-by-step, potentially backtracking and self-correcting (this is where the extra compute goes)
3. **Final answer** — a polished response generated after reasoning is complete
4. **RL reward loop** (during training only) — correct answers reinforce the reasoning patterns that produced them

Test-time compute: the mechanics

Snell et al. (2024): 'Scaling LLM Test-Time Compute'

Snell et al. (2024) showed that **additional compute at inference** improves performance predictably — and on some problems, a small model thinking longer outperforms a large model thinking quickly.

Two mechanisms for spending inference compute

Mechanism	How it works	When it helps
Process-reward models	Train a verifier to score each reasoning step; generate many solutions; pick the one with highest step-level scores	Problems with clear intermediate steps (math proofs)
Adaptive self-revision	The model critiques and refines its own output iteratively; RL trains it to know when to keep thinking	Open-ended problems where the model can self-evaluate

Compute-optimal scaling improves efficiency **2–4×** over naive approaches (e.g., just generating more samples).

The s1 experiment: reasoning is surprisingly simple

Muennighoff et al. (2025)

s1 showed that test-time scaling can be achieved with remarkably little effort:

1. Fine-tune Qwen2.5-32B on just **1,000 curated examples** (the "s1K" dataset)
2. At inference, apply **budget forcing**: append the word "Wait" to force continued reasoning
3. Result: Exceeds o1-preview on AIME 2024 by up to **27%**

The implication

You don't need massive RL infrastructure to get reasoning capabilities. A small amount of high-quality reasoning data + a simple inference trick can unlock substantial test-time scaling. This suggests reasoning is latent in large pretrained models — it just needs to be **activated**.

Claude's extended thinking

Anthropic (February 2025 — present)

Anthropic implemented reasoning as a **toggle within a single model** — same weights, different inference behavior. Unlike OpenAI, Claude's thinking tokens are **visible** to developers.

API usage (see companion notebook)

```
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=16000,
    thinking={"type": "enabled", "budget_tokens": 10000},
    messages=[{"role": "user", "content": "Prove that √2 is irrational."}]
)
# Response includes a visible "thinking" block + final answer
```

Key differences from OpenAI

Feature	OpenAI o-series	Claude extended thinking
Thinking visibility	Hidden	**Visible**
		Budget tokens (1K–128K) or

The frontier landscape: early 2026

Where we stand

Model	Developer	Key capability
Claude Opus 4.6	Anthropic	1M context, adaptive thinking, visible reasoning
GPT-5	OpenAI	Native multimodal, 94.6% AIME
Gemini 2.5 Pro	Google	1M context, built-in thinking
DeepSeek-R1	DeepSeek	Open-weight reasoning, 671B MoE, $5.5M training
Llama 4 Maverick	Meta	Open-weight, 400B MoE, 1M context
Qwen 3	Alibaba	89.7% AIME, open-weight, 235B

The MoE + reasoning convergence

Benchmark saturation and the reasoning gap

Where thinking helps — and where it doesn't

Benchmark	Best score	Human	Status
MATH-500	98.0%	—	Saturated — thinking solves it
AIME 2025	92.7%	—	Near-saturated — thinking solves it
GPQA Diamond (PhD science)	94.3%	~65% expert	Near-saturated
SWE-bench Verified (coding)	80.9%	—	Active — but improving fast
ARC-AGI-2 (novel reasoning)	**4.4%**	**60%**	**Not saturated** — 20× gap
Humanity's Last Exam	**48.1%**	**~90%**	**Not saturated** — progress stalling

The key pattern

Thinking dramatically helps on problems that *decompose* into verifiable steps (math, coding). It helps less on problems requiring **novel abstractions** (ARC-AGI-2) or **deep domain expertise** (HLE). More thinking

Discussion: what does it mean to "think"?

The deepest question in this course

1. **Thinking or performing?** When o3 generates a 10,000-token reasoning trace to solve a math problem, is it *thinking* — or producing a sophisticated pattern that *looks like* thinking? How would you test the difference? (Revisit Lecture 1: is ChatGPT conscious?)
2. **The DeepSeek-R1 emergence:** R1-Zero developed self-verification and backtracking *without being taught these behaviors*. The model was rewarded only for correct answers — yet it learned to doubt itself, re-examine its work, and try alternative approaches. Is this "just optimization" — or is something deeper happening?
3. **The computational argument:** Merrill & Sabharwal (2024) proved that CoT makes transformers Turing-complete. Does this formal

Further reading

Further reading

Wei et al. (2022, *NeurIPS*) "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models" — Where it all started.

Snell et al. (2024, *arXiv*) "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters" — The theoretical foundation for inference-time scaling.

DeepSeek-AI (2025, *arXiv*) "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning" — Open-weight reasoning model; GRPO algorithm.

Muennighoff et al. (2025, *arXiv*) "s1: Simple Test-Time Scaling" — 1,000 examples + "Wait" trick beats o1-preview.

Merrill & Sabharwal (2024, *arXiv*) "The Expressive Power of Transformers with Chain of Thought" — CoT makes transformers Turing-complete.

Questions?

Email

Discord

Office hours

Up next...

Agents, tools, and the agentic era: when LLMs start acting in the world